

TRAINEE LAB GUIDE

Shopify Agentic AI Developer Bootcamp — Koenig Solutions for V Group Inc.

Lab 3.2

Create a Custom App in the Dev Dashboard & Generate a Scoped Admin API Token

Module	3 — Shopify Platform, Storefront API, Admin API & Native AI
Topic	3.2 — Authentication, OAuth & Access Scopes
Estimated time	35–45 minutes
Tools needed	Terminal, Node.js 20+, Shopify CLI, a code editor, a Shopify Partner account with a dev store
Prerequisite	Topic 3.1 (Shopify architecture) on the program website; a Partner account and dev store already created

What You're Building

GOAL

A custom app created in Shopify's **Dev Dashboard**, linked to a local Node.js project with the **Shopify CLI**, with its Admin API scopes declared in a `shopify.app.toml` file and deployed — installed on your dev store, with a working helper that exchanges its Client ID/secret for a short-lived Admin API access token.

There's no checkbox screen for Admin API scopes anywhere in this flow — that surprises most people coming from older Shopify tutorials. Scopes live in a text file in your own project, edited by hand and pushed live with a CLI command. This lab walks through that exact mechanism, in order.

Steps 1–3: Create the App & Declare Scopes

1 Create the app in the Dev Dashboard

In your dev store's Admin, go to **Settings** → **Apps** → **Develop apps**, then click **Build apps in Dev Dashboard**. In the Dev Dashboard, click **Create app**, name it (e.g. "Agentic AI Bootcamp"), and confirm.

What you'll see: the app now has an initial version (e.g. "agentic-ai-bootcamp-1") under its **Versions** tab — not yet linked to any local project folder, and not yet installed on any store. Unlike the old flow, it doesn't ask you to pick a target store at creation time; that happens in Step 5.

2 Install the Shopify CLI and link a local project

TERMINAL (ONE-TIME CLI INSTALL)

```
npm install -g @shopify/cli@latest
```

TERMINAL

```
mkdir shopify-admin-agent
cd shopify-admin-agent
shopify app config link
```

`shopify app config link` runs an interactive flow: it opens a browser-based device login (prints a one-time code and a link to approve it), asks which Partner organization to use, then asks "Create this project as a new app on Shopify?" — choose **"No, connect it to an existing app"** and select the app from Step 1.

What you'll see: a new `shopify.app.toml` file in the folder, pre-filled with the app's Client ID and a blank `scopes` field, plus a terminal message confirming `shopify.app.toml` is now linked to `"..."` on Shopify .

3 Declare Admin API scopes in `shopify.app.toml`

Open the generated file and edit its `[access_scopes]` block:

SHOPIFY.APP.TOML — [ACCESS_SCOPES]

```
[access_scopes]
scopes = "read_products,read_orders,write_draft_orders,read_customers"
```

Resist listing everything "just in case." These four scopes are exactly what this module's labs need — a product-search tool doesn't need `write_orders` , and granting it anyway is the over-scoping habit Topic 3.5 warns about.

Steps 4–6: Deploy, Install & Request Data Access

4 Deploy the new version

TERMINAL (INSIDE THE PROJECT FOLDER)

```
shopify app deploy
```

Confirm "**Yes, release this new version**" when prompted.

What you'll see: a confirmation ending in something like `New version released to users. agentic-ai-bootcamp-2 [1] .` Editing `shopify.app.toml` alone changes nothing on Shopify's side — every edit needs a fresh `shopify app deploy` to actually take effect.

5 Install the app on your dev store

Back in the Dev Dashboard, open the app's **Overview** tab, click **Install app**, and select your dev store — it must belong to the same Partner organization as the app. This redirects to the store's own Admin with a consent screen listing what the app needs access to, translated into plain language from the scopes declared in Step 3. Click **Install** to confirm.

What you'll see: back in the Dev Dashboard's Overview tab, the **Installs** counter now reads `1` .

6 Request Protected customer data access

Any Admin API object touching customer PII — orders, customers — sits behind a second, separate approval, even for an app you own on your own dev store. This screen doesn't live in the Dev Dashboard at all — go to `partners.shopify.com` instead, open your app, click **API access requests** in the sidebar, then under **Protected customer data access** click **Request access**. Check the box, choose applicable reasons (e.g. "Store management," "App functionality"), and Save.

Easy to miss: skip this step and an otherwise-correct `orders` or `customers` query fails with an approval-style error — the token and scopes are fine, this gate is just separate, and it's on a different site than everything else in this lab. For a dev-store-only app like this one, no review/submission step is required.

Steps 7–8: Retrieve Credentials & Exchange Them for a Token

7 Copy your Client ID and Client secret

In the Dev Dashboard, open the app's **Settings** tab → **Credentials**. Copy the Client ID and Client secret into a `.env` file in your project:

.ENV

```
SHOPIFY_STORE_DOMAIN=your-store.myshopify.com
SHOPIFY_APP_CLIENT_ID=your-client-id
SHOPIFY_APP_CLIENT_SECRET=your-client-secret
```

.GITIGNORE

```
node_modules
.env
```

Treat the Client secret like a password. It's hidden behind an eye icon in the Dev Dashboard with a "Rotate" button — if it's ever pasted somewhere outside your own machine, rotate it before continuing.

8 Write a token-exchange helper and verify it

Unlike the old legacy-app flow, nothing here displays a ready-to-copy access token. Write a small helper that exchanges the Client ID/secret for one programmatically, caching it until it's close to expiry:

ADMINCLIENT.JS

```
require("dotenv").config();
const domain = process.env.SHOPIFY_STORE_DOMAIN;
const clientId = process.env.SHOPIFY_APP_CLIENT_ID;
const clientSecret = process.env.SHOPIFY_APP_CLIENT_SECRET;
let cachedToken = null, cachedTokenExpiresAt = 0;

async function getAdminAccessToken() {
  if (cachedToken && Date.now() < cachedTokenExpiresAt) return cachedToken;
  const res = await fetch(`https://${domain}/admin/oauth/access_token`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      client_id: clientId,
      client_secret: clientSecret,
      grant_type: "client_credentials",
    }),
  });
  const data = await res.json();
```

```

if (!data.access_token) throw new Error("Token exchange failed: " + JSON.stringify(data));
cachedToken = data.access_token;
cachedTokenExpiresAt = Date.now() + (data.expires_in - 60) * 1000;
return cachedToken;
}

async function adminRequest(query, variables) {
  const token = await getAdminAccessToken();
  const res = await fetch(`https://${domain}/admin/api/2026-07/graphql.json`, {
    method: "POST",
    headers: { "Content-Type": "application/json", "X-Shopify-Access-Token": token },
    body: JSON.stringify({ query, variables }),
  });
  const json = await res.json();
  if (json.errors) throw new Error(JSON.stringify(json.errors));
  return json.data;
}

module.exports = { adminRequest };

```

TEST-ADMIN.JS

```

const { adminRequest } = require("./adminClient");
adminRequest(`query { shop { name email } }`)
  .then((data) => console.log(JSON.stringify(data, null, 2)))
  .catch(console.error);

```

TERMINAL

```

npm install dotenv
node test-admin.js

```

What you'll see: JSON printing your dev store's real `name` and `email`. If you skipped Step 6 and try an `orders` or `customers` query instead, expect a real `shop` result but an approval-style error specifically on that field — confirming the token itself is correct and isolating the problem to the Protected Customer Data gate.

Acceptance checklist

- ☐ App created in the Dev Dashboard and linked to a local project via `shopify app config link`, within your own Partner organization
- ☐ Only the scopes actually needed for this module are declared in `shopify.app.toml` and deployed
- ☐ Protected customer data access requested and approved at `partners.shopify.com`, so `read_orders` / `read_customers` queries don't fail with an approval error
- ☐ Client ID and secret are stored in `.env`, never committed to git

- ☐ Your token-exchange helper successfully authenticates a test GraphQL request, and re-requests a token once the cached one is near expiry

Dev store only. This token can create draft orders and read customer data — never point it at a production store, even out of curiosity.

Troubleshooting

Symptom	Likely cause	Try this
An <code>orders / customers</code> query fails with an approval-style error even though scopes look right	Protected customer data access not yet requested/approved	Complete Step 6 at <code>partners.shopify.com</code> — this is a separate gate from the scopes, and lives on a different site than the Dev Dashboard
<code>shopify app config link</code> can't find your app, or links to the wrong one	Wrong Partner organization selected, or the app wasn't created in the Dev Dashboard first	Confirm Step 1 was completed before linking, and double-check which organization you selected during login
Editing scopes in <code>shopify.app.toml</code> seems to have no effect	Forgot to run <code>shopify app deploy</code> after editing	Every edit to <code>shopify.app.toml</code> needs a fresh deploy before it takes effect on Shopify's side
Token exchange returns an error, or no <code>access_token</code> field	Client ID/secret copied incorrectly, or the app isn't installed on that store yet	Re-copy credentials from Settings → Credentials; confirm the Overview tab's Installs counter reads at least 1
Requests work for a while, then start failing after about a day	A cached token is being reused past its <code>expires_in</code> window (24 hours)	Confirm the helper checks <code>cachedTokenExpiresAt</code> and requests a fresh token before the old one lapses

Where This Goes Next

This Client ID/secret and the client-credentials token-exchange helper are the Admin-side credential used for the rest of Module 3 — Topic 3.4's draft orders and Topic 3.6's tool-wrapping lab both reuse `adminClient.js` as-is — and again in Module 5's admin-scoped agent, where only the declared scopes in `shopify.app.toml` grow to cover more of the store.